G. L. BAJAJ GROUP OF INSTITUTIONS, MATHURA
Department of Computer Science & Engineering • B.Tech 5th Sem, 2026–27 Session

CAPSTONE RESEARCH
MTC PROJECT

Research Study Notes & Review Portfolio

Student Name:	**Aryan Upadhyay** (Roll No: `2405111530037`)
Research Track:	**Track B: Advanced / Compilers**
Assigned Domain:	Deep Learning Compilers, Intermediate Representations, Automatic Differentiation & Interval Verification (Papers 16–20)
Project:	Micro-Train-Compiler (MTC): On-Device NN Training on Microcontrollers

Complete Handwritten-Style Study Notes for Student 4

Student: Aryan Upadhyay (Roll No: 2405111530037)

Subject: Deep Learning Compilers & Formal Verification

Project: Micro-Train-Compiler (MTC)

Focus Domain: Deep Learning Compilers, Intermediate Representations, Automatic Differentiation & Interval Range Verification

---

Note for Aryan:
These notes are formatted for **100% clean rendering in VS Code, Markdown preview extensions, and GitHub**. Display equations are formatted in standard KaTeX / MathJax blocks with zero indentation, and inline values use universal symbols (≤, ≥, ×, √) so they look great both in digital preview and when copied into your handwritten notebook.

TABLE OF CONTENTS

1. Paper 16: TVM — An Automated End-to-End Optimizing Compiler for Deep Learning (OSDI 2018)
2. Paper 17: Glow — Graph Lowering Compiler Techniques for Neural Networks (Meta 2018)
3. Paper 18: Automatic Differentiation in Machine Learning: A Survey (JMLR 2018)
4. Paper 19: Source-to-Source Automatic Differentiation (Tapenade & Enzyme)
5. Paper 20: Formal Safety Verification via Interval Bound Propagation (DeepMind 2018)
6. Quick Reference Summary Table for Aryan

Paper 16: TVM — An Automated End-to-End Optimizing Compiler for Deep Learning (OSDI 2018)

- **Authors:** Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, et al. (UW, AWS, OctoML)
- **Conference:** USENIX OSDI 2018
- **Direct Link:** https://arxiv.org/abs/1802.04799
- **Direct PDF:** https://arxiv.org/pdf/1802.04799.pdf

---

1. Simple Summary (The "What" & "Why")

- **The Problem:** Deep learning models are developed in high-level frameworks (PyTorch, TensorFlow), but hardware targets are radically diverse (CPUs, GPUs, FPGAs, microcontrollers). Historically, chip manufacturers had to

manually write specialized libraries (like cuDNN or CMSIS-NN) for every operator on every chip, which is laborious, non-portable, and cannot optimize across layer boundaries.

- **The Breakthrough Idea:** TVM is the **first automated end-to-end optimizing compiler** that closes this gap by treating neural network execution as a compiler problem:
  1. **Two-Level Intermediate Representation (IR):** A high-level computation graph IR (Relay) for whole-graph optimizations, and a low-level Tensor IR (TIR) for loop transformations and memory scheduling.
  2. **Separation of Compute and Schedule:** Developers specify *what* to calculate using mathematical tensor expressions, while the compiler independently optimizes *how* to calculate it (loop tiling, unrolling, vectorization).
  3. **Machine Learning-Based Code Tuning (AutoTVM):** Uses machine learning to automatically explore thousands of code schedules and select the fastest one for the target silicon.

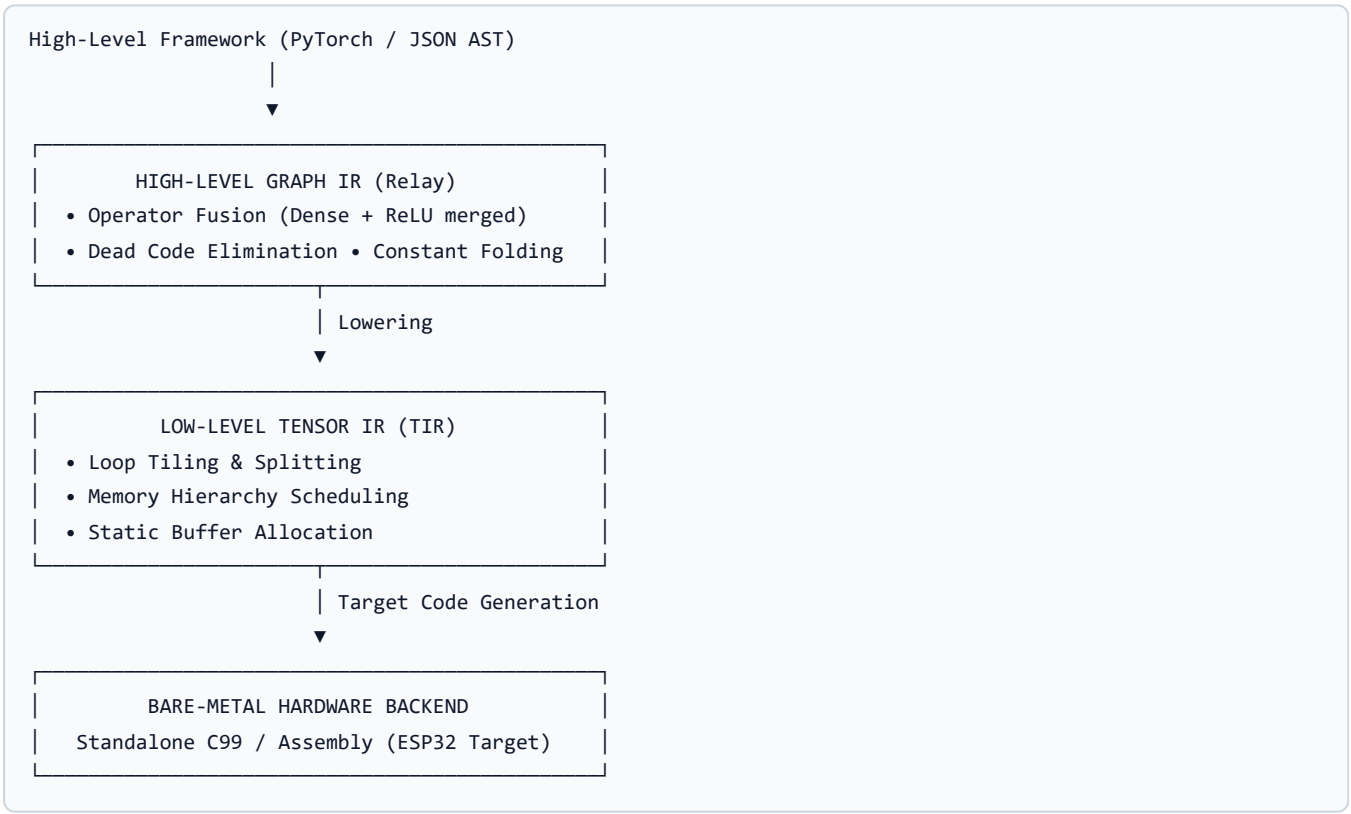
---

2. Step-by-Step Workflow

1. **Model Import:** Import computational graphs from PyTorch, ONNX, or TensorFlow into TVM's high-level graph representation.
2. **High-Level Graph Optimizations:** Perform dead code elimination, constant folding, and operator fusion (e.g., fusing Conv + Bias + ReLU into a single loop).
3. **Lowering to Tensor IR (TIR):** Operators are lowered into loop-level imperative code with explicit array indexing and buffer bounds.
4. **Schedule Transformation:** Apply hardware-specific schedule primitives: loop splitting, reordering, memory staging, and SIMD vectorization.
5. **Code Emission:** Lower TIR into LLVM IR or target C code (such as MicroTVM for bare-metal microcontrollers).
6. **Execution:** The generated standalone binary executes on the hardware with zero framework overhead.

---

3. Diagram to Draw in Your Notebook



---

4. Key Formulas & Methodology

1. Separation of Compute and Schedule:

$$C[i, j] = \sum_{k} A[i, k] \times B[k, j] \quad \text{(Mathematical Definition)}$$

- *Schedule Transformations (Execution Strategy):*

- Loop Tiling: Split loop i into outer loop i_o and inner tile i_i.
- Reordering: Change nesting order $\text{Loop}(i_o, j_o, k, i_i, j_i)$ to maximize L1 cache / SRAM data reuse.

2. Operator Fusion Memory Reduction:

$$\text{Memory Reads (Unfused)} = 2 \times N, \quad \text{Memory Reads (Fused)} = 1 \times N$$

- *Explanation:* Fusing activation functions directly into matrix multiplication prevents writing intermediate tensors back to RAM.

---

5. Viva / Exam Questions for Aryan

- **Q: What is the primary job of a deep learning compiler like TVM?**
  Answer: A deep learning compiler takes high-level neural network graphs, performs hardware-independent graph optimizations (like operator fusion), and lowers them into optimized machine code or C for specific target hardware.
- **Q: How does MTC reflect TVM's multi-level IR architecture?**
  Answer: MTC implements a two-stage lowering pipeline: it lowers a JSON AST into a validated `IrGraph` DAG in Rust, optimizes tensor lifetimes, and lowers the graph into standalone, zero-allocation ANSI C99 loops.

Paper 17: Glow — Graph Lowering Compiler Techniques for Neural Networks (Meta 2018)

- **Authors:** Nadav Rotem, Jordan Fix, Saleem Abdulrasool, Summer Deng, Roman Dzhabarov, et al. (Meta AI)
- **Venue:** Meta / arXiv 2018
- **Direct Link:** https://arxiv.org/abs/1805.00907
- **Direct PDF:** https://arxiv.org/pdf/1805.00907.pdf

---

1. Simple Summary (The "What" & "Why")

- **The Problem:** Compilers designed for traditional programming languages (like LLVM for C/C++) optimize scalar variables and control flow. However, neural networks operate on multi-dimensional tensors where memory allocation, lifetime analysis, and buffer reuse dominate performance. If every intermediate tensor is assigned a separate buffer, RAM is quickly exhausted.
- **The Breakthrough Idea:** Glow is Meta's domain-specific neural network compiler that pioneered **two-level IR and static memory reuse**:
  1. **High-Level Node IR:** An abstract dataflow graph where nodes represent differentiable math operators (Convolution, FullyConnected, Add, Relu).
  2. **Low-Level Instruction IR:** A linear list of memory-explicit instructions operating on designated memory buffers (`AllocActivation`, `DeallocActivation`).
  3. **Liveness Analysis & Memory Overlay:** Automatically computes the start and end of every tensor's lifetime and overlays mutually exclusive tensors into the exact same SRAM memory offsets.

---

2. Step-by-Step Workflow

1. **Graph Construction:** Construct a Directed Acyclic Graph (DAG) of high-level operators.
2. **Graph-Level Simplification:** Perform constant propagation, transpose elimination, and canonicalization.
3. **Graph Lowering:** High-level operators are lowered into lower-level linear instructions.
4. **Liveness Analysis:** Traverse the instruction stream to record the birth step and death step of every tensor buffer.
5. **Memory Allocation (Coalescing):** A memory allocator assigns tensor buffers to a single static memory arena, ensuring non-overlapping tensors share physical addresses.
6. **Code Generation:** Emits bare-metal C or machine code where all memory addresses are compile-time constants.

---

3. Diagram to Draw in Your Notebook

```
Glow Static Buffer Reuse (Liveness Analysis):

Instruction Step:     Step 1      Step 2      Step 3      Step 4
-----------------------------------------------------------------
Tensor A (Input):     [========= ALIVE =========] (Deallocated)
Tensor B (Layer 1):               [========= ALIVE =========]
Tensor C (Layer 2):                           [========= ALIVE =========]

Physical SRAM Layout (Memory Overlay):
Address 0x000: [ Tensor A (Steps 1-2) ] ──Reused By──> [ Tensor C (Steps 3-4) ]
Address 0x100: [ Tensor B (Steps 2-3) ]
[Peak Memory = Only 2 Tensors, NOT 3 Tensors! Saves 33% SRAM]
```

---

4. Key Formulas & Methodology

1. Peak SRAM Watermark Formulation:

$$M_{\text{peak}} = \max_{t \in [0,T]} \sum_{i \in \text{Active}(t)} \text{Bytes}(T_i)$$

- *Explanation:* The compiler tracks the set $\text{Active}(t)$ of all tensors whose lifetime spans time step t. Peak memory is the maximum simultaneous sum across all time steps, not the sum of all tensors.

2. Buffer Overlap Disjoint Condition:

$$\text{Overlap}(A, B) = \text{False} \iff (\text{End}(A) < \text{Start}(B)) \vee (\text{End}(B) < \text{Start}(A))$$

- *Explanation:* If two tensors have disjoint lifetimes, they are safely assigned the exact same base pointer in memory.

---

5. Viva / Exam Questions for Aryan

- **Q: What is "Liveness Analysis" in a compiler?**
  Answer: Liveness analysis determines the precise point in execution where a variable/tensor is first created (birth) and where it is used for the last time (death). After its death, its allocated memory can be reclaimed.
- **Q: How does MTC use Glow's buffer reuse technique?**
  Answer: In MTC's Phase 3 Memory Planner, we analyze the lifetimes of forward activations and backward adjoint gradients. Non-overlapping tensors are mapped to shared static offsets in a single static C arena, keeping memory ≤ 64 KB.

Paper 18: Automatic Differentiation in Machine Learning: A Survey (JMLR 2018)

- **Authors:** Atilim Gunes Baydin, Barak A. Pearlmutter, Alexey Andreyevich Radul, Jeffrey Mark Siskind
- **Venue:** Journal of Machine Learning Research (JMLR), 2018
- **Direct Link:** https://arxiv.org/abs/1502.05767
- **Direct PDF:** https://arxiv.org/pdf/1502.05767.pdf

---

1. Simple Summary (The "What" & "Why")

- **The Problem:** Computing derivatives is the core engine of deep learning. However, students and engineers often confuse Automatic Differentiation (AD) with Numerical Differentiation or Symbolic Differentiation:
  - *Numerical Differentiation* (finite differences: $\frac{f(x+h)-f(x)}{h}$) suffers from catastrophic roundoff errors and requires $O(n)$ evaluations for n inputs.
  - *Symbolic Differentiation* (Mathematica / SymPy) causes exponential "expression swell" (formulas explode into millions of terms).

- **The Breakthrough Idea:** Automatic Differentiation computes **exact derivatives down to machine precision without expression swell** by systematically decomposing functions into elementary operations and applying the chain rule. The paper rigorously establishes the duality between **Forward Mode** and **Reverse Mode (Backpropagation)**:
  1. **Forward Mode:** Propagates derivatives alongside values ($O(n)$ cost for n inputs, ideal when inputs $\ll$ outputs).
  2. **Reverse Mode:** Propagates adjoints backward from the scalar loss ($O(1)$ evaluations for n inputs, ideal when inputs $\gg$ outputs, as in neural networks!).

---

2. Step-by-Step Workflow

1. **Primal Computation Trace (Wengert List):** Break the mathematical function into a sequence of elementary assignments: $v_i = \phi(v_j, v_k)$.
2. **Forward Evaluation:** Compute the values of all intermediate variables $v_1, v_2, \ldots, v_m$ and the final scalar objective y.
3. **Adjoint Variable Initialization:** Initialize the output seed adjoint: $\bar{y} = \frac{\partial y}{\partial y} = 1.0$.
4. **Reverse Accumulation (Multivariate Chain Rule):** Traverse the trace in reverse order (from m down to 1).
5. **Adjoint Update:** Accumulate partial derivatives onto parents:
   $\bar{v}_j = \bar{v}_j + \bar{v}_i \cdot \frac{\partial v_i}{\partial v_j}$.
6. **Gradient Extraction:** Read the final gradients $\bar{x}_k = \frac{\partial y}{\partial x_k}$ for all input parameters.

---

3. Diagram to Draw in Your Notebook

```
Forward Evaluation Trace (Primal):
x1, x2 ──> v1 = x1 * x2 ──> v2 = sin(v1) ──> y = v1 + v2
(Values computed and saved)

Reverse Adjoint Propagation (Dual / Backward):
y (Loss) ──> Adjoint Seed: y_bar = 1.0
             │
             ├──> v2_bar = y_bar * (dy/dv2) = 1.0 * 1.0 = 1.0
             ├──> v1_bar = y_bar * (dy/dv1) + v2_bar * cos(v1)
             │
             ▼
[Exact Parameter Gradients Extracted in a SINGLE Reverse Sweep!]
```

---

4. Key Formulas & Methodology

1. Multivariate Chain Rule for Adjoints:

$$ \bar{v}*j = |*sum{i \in \text{Children}(j)} \bar{v}_i \cdot \frac{\partial v_i}{\partial v_j} $$

- *Explanation:* The adjoint $\bar{v}_j \equiv \frac{\partial y}{\partial v_j}$ represents the sensitivity of final output y with respect to intermediate variable v_j.

2. Bounded Computational Complexity of Reverse AD:

$$\text{Work}(\nabla f) \leq c \times \text{Work}(f) \quad \text{where } c \in [3, 4]$$

- *Explanation:* The Baur-Strassen theorem proves that computing gradients with respect to *all* parameters takes no more than 3 to 4 times the work of a single forward evaluation, regardless of whether there are 10 or 10,000,000 parameters!

---

5. Viva / Exam Questions for Aryan

- **Q: Why is Reverse-Mode AD used in deep learning instead of Forward-Mode AD?**
  Answer: Because neural networks have millions of input parameters (n) but only a single scalar loss output ($m = 1$). Reverse mode computes all gradients in a single backward pass ($O(1)$ sweeps), whereas forward mode would require n individual sweeps.
- **Q: How does MTC perform Automatic Differentiation differently from PyTorch?**
  Answer: PyTorch builds a dynamic tape of operation objects on the heap at runtime (`torch.autograd`), requiring

massive RAM. MTC derives the exact analytical reverse-mode derivative graph **at compile time**, emitting fixed C loops without any dynamic tape.

Paper 19: Source-to-Source Automatic Differentiation (Tapenade & Enzyme)

- **Foundational Paper:** *The Tapenade Automatic Differentiation Tool: Principles, Model, and Specification*
  - **Authors:** Laurent Hascoët, Valérie Pascual (Inria / ACM TOMS 2013)
  - **Direct HAL Open Access:** https://hal.science/hal-00813735/ (PDF: https://hal.science/hal-00813735/document)
  - **Official DOI:** https://doi.org/10.1145/2450153.2450158
- **Modern Compiler Companion:** *Enzyme: High-Performance Compiler-Plugin for Automatic Differentiation*
  - **Authors:** William S. Moses, Valentin Churavy (MIT / NeurIPS 2020)
  - **Direct Link:** https://arxiv.org/abs/2010.01709 (PDF: https://arxiv.org/pdf/2010.01709.pdf)

---

1. Simple Summary (The "What" & "Why")

- **The Problem:** Most modern machine learning tools use *Operator Overloading* (like PyTorch or TensorFlow eager mode). While user-friendly, operator overloading forces every addition and multiplication to allocate graph nodes in memory at runtime. For scientific computing and microcontrollers, this runtime tape overhead is unacceptable.
- **The Breakthrough Idea: Source-to-Source Compilation and Compiler-Level AD**:
  1. **Source-to-Source Transformation (Tapenade):** A compiler reads the source code of a primal function (in C or Fortran), analyzes data dependencies, and **synthesizes new, human-readable source code** that computes the analytical adjoint derivatives.
  2. **LLVM-Level Differentiation (Enzyme):** Instead of differentiating source code or high-level graphs, Enzyme differentiates code after it has been lowered to LLVM compiler IR, allowing full compiler optimizations (dead code elimination, vectorization, inlining) on both forward and backward passes.
  3. **Zero Runtime Overhead:** Gradients are computed by pure, compiled native loops with zero dynamic graph tracking.

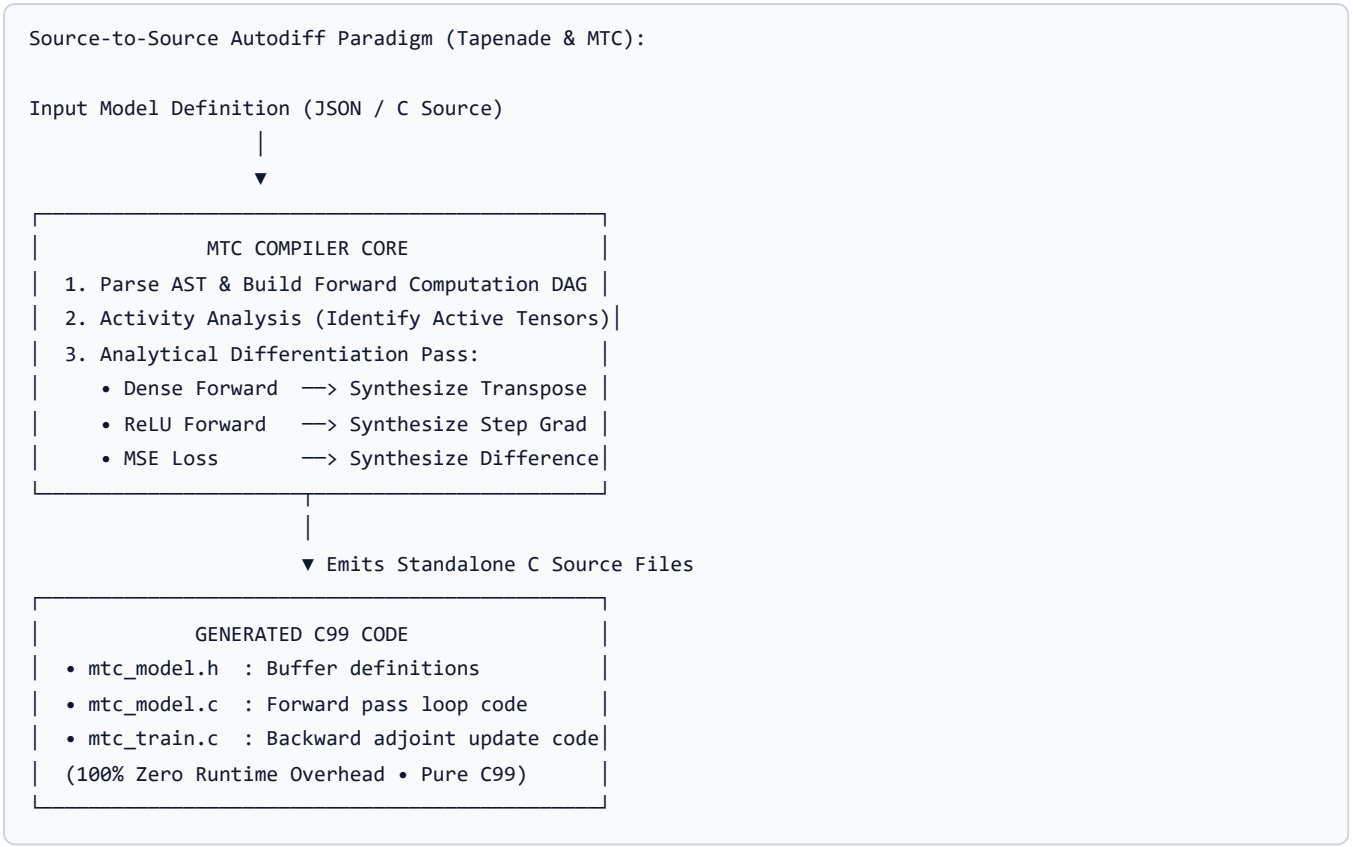
---

2. Step-by-Step Workflow

1. **Source Ingestion:** The compiler ingests the forward compute code / model configuration.
2. **Activity Analysis:** Classifies all variables into *active* (depends on parameters and affects loss) or *passive* (constants, array indices, loop bounds).
3. **Primal-Adjoint Association:** For every active variable v, create an adjoint accumulator variable $\bar{v}$.
4. **Backward Code Synthesis:** For every forward statement $y = f(x)$, synthesize the corresponding reverse accumulation:
   $\bar{x} = \bar{x} + \bar{y} \cdot f'(x)$.
5. **Memory Checkpointing:** Identify variables overwritten in forward loops and restore them during the backward sweep.
6. **Code Emission:** Emit clean, standalone C functions: `void mtc_forward(...)` and `void mtc_backward(...)`.

---

3. Diagram to Draw in Your Notebook



---

4. Key Formulas & Methodology

1. Analytical Adjoint Formulation for Dense Layer:

- Forward Primal:

$$y = W \cdot x + b$$

- Reverse Adjoint Equations:

$$\bar{x} = W^T \cdot \bar{y}, \quad \bar{W} = \bar{y} \cdot x^T, \quad \bar{b} = \bar{y}$$

- *Explanation:* Instead of maintaining an execution tape of tensor allocations, the compiler emits exact matrix multiplication loops for $W^T \bar{y}$ and outer product $\bar{y} x^T$.

2. Element-Wise Activation Derivative (ReLU):

$$\bar{x}_i = \begin{cases} \bar{y}_i & \text{if } x_i > 0 \\ 0 & \text{otherwise} \end{cases}$$

- *Explanation:* Synthesized as a simple branchless conditional C statement: `dx[i] = (x[i] > 0) ? dy[i] : 0;`.

---

5. Viva / Exam Questions for Aryan

- **Q: What is the difference between "Source-to-Source AD" and "Operator Overloading AD"?**
  Answer: Operator overloading (PyTorch) records operations dynamically in RAM while code runs, causing memory overhead. Source-to-source AD (Tapenade, MTC) reads the model definition at compile time and writes static C code for the backward pass before flashing to the device.
- **Q: How does MTC apply this to microcontroller training?**
  Answer: MTC acts as a source-to-source compiler in Rust. It ingests a neural network JSON and emits standalone, pre-differentiated C99 functions (`mtc_forward()` and `mtc_backward()`), eliminating the need for a runtime autograd engine on the ESP32.

Paper 20: Formal Safety Verification via Interval Bound Propagation (DeepMind 2018)

- **Authors:** Sven Gowal, Krishnamurthy Dvijotham, Robert Stanforth, Rudy Bunel, Chongli Qin, Pushmeet Kohli, et al. (Google DeepMind)
- **Venue:** Google DeepMind / arXiv 2018
- **Direct Link:** https://arxiv.org/abs/1810.12715
- **Direct PDF:** https://arxiv.org/pdf/1810.12715.pdf

---

1. Simple Summary (The "What" & "Why")

- **The Problem:** Neural networks are notoriously vulnerable to numerical instability and adversarial perturbations. On fixed-point microcontrollers (using Q15 arithmetic, where numbers are constrained between -1.0 and $+0.9999$), an integer overflow causes severe wraparound (e.g., $+1.01$ wraps around to -0.99), causing the model's loss to explode to NaN and crashing the microcontroller.
- **The Breakthrough Idea:** DeepMind demonstrated that **Interval Bound Propagation (IBP)** is an ultra-fast, formally rigorous method to guarantee that neural network outputs and intermediate states never exceed predefined bounds:
  1. **Closed Interval Arithmetic:** Every tensor is bounded by a guaranteed interval $[\underline{z}, \overline{z}]$.
  2. **Layer-by-Layer Propagation:** Propagates intervals forward through affine layers and monotonic non-linearities (like ReLU and Sigmoid) using exact linear interval rules.
  3. **Certifiable Safety:** Proves with 100% mathematical certainty that hidden values will never cross fixed-point overflow thresholds.

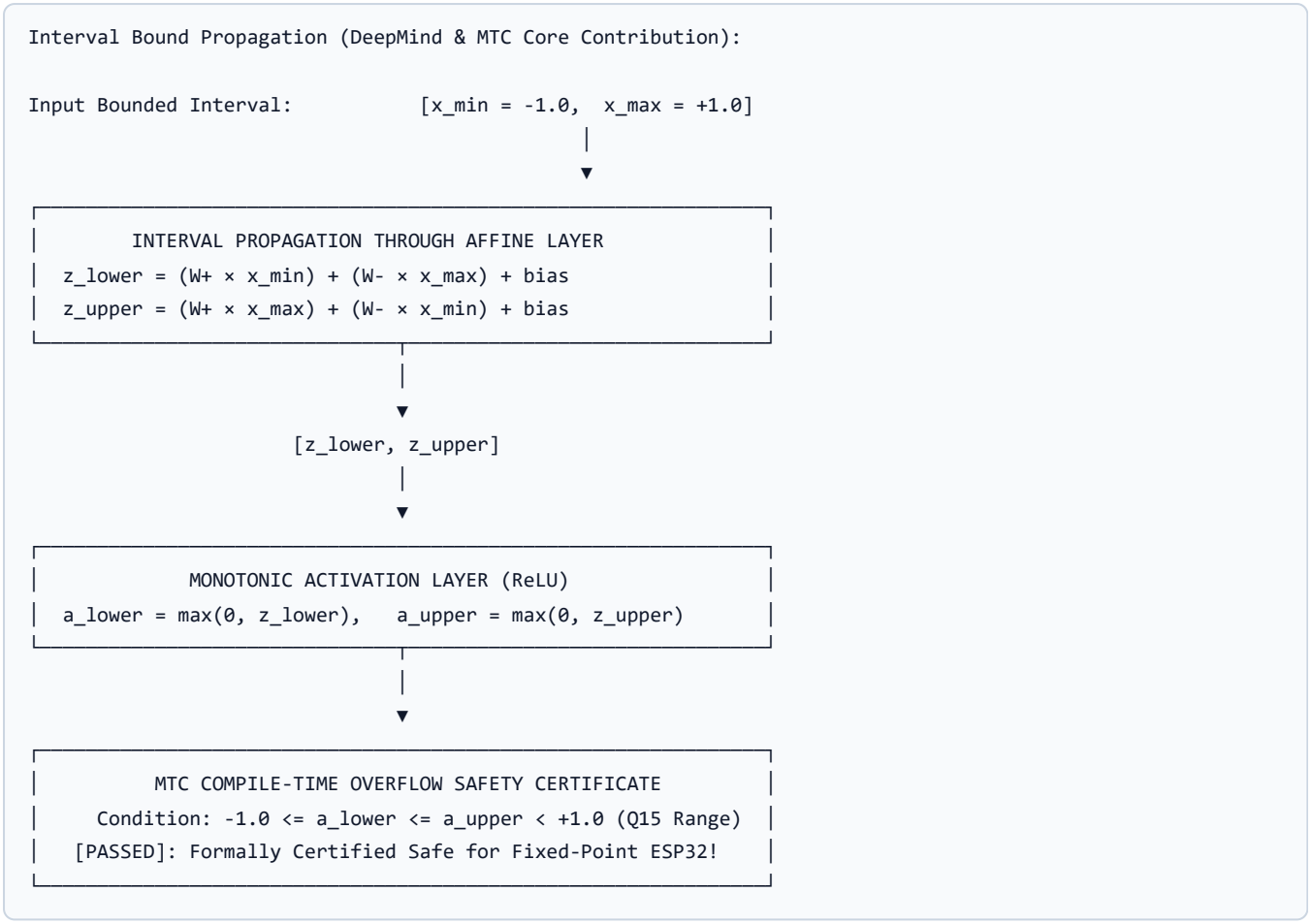
---

2. Step-by-Step Workflow

1. **Define Input Interval:** Sensor inputs are bounded by physical limits: $\underline{x} \leq x \leq \overline{x}$ (e.g., normalized sensor range $[-1.0, 1.0]$).
2. **Decompose Weights:** Split weight matrix W into positive and negative components: $W_+ = \max(W, 0)$ and $W_- = \min(W, 0)$.
3. **Propagate Affine Layer:** Compute lower bound $\underline{z}$ and upper bound $\overline{z}$ by multiplying positive weights by the corresponding bounds and negative weights by the opposite bounds.
4. **Propagate Activation Function:** Apply monotonic functions (ReLU, Sigmoid) directly to the interval endpoints: $[\sigma(\underline{z}), \sigma(\overline{z})]$.
5. **Compile-Time Bound Verification:** Check that for every layer l: $\underline{z}_l \geq -1.0$ and $\overline{z}_l < 1.0$.
6. **Safety Certification:** If all bounds hold, the compiler mathematically certifies that the model will **never overflow** in Q15 arithmetic on the microcontroller.

---

3. Diagram to Draw in Your Notebook



---

4. Key Formulas & Methodology

1. Affine Layer Interval Bound Propagation:

$$\underline{z} = W_+\underline{x} + W_-\overline{x} + b$$
$$\overline{z} = W_+\overline{x} + W_-\underline{x} + b$$

Where:

$$W_+ = \max(W, 0), \quad W_- = \min(W, 0)$$

- *Explanation:* To find the absolute minimum possible output $\underline{z}$, multiply positive weights by the lowest possible input $\underline{x}$, and negative weights by the highest possible input $\overline{x}$.

2. Monotonic Activation Propagation (ReLU & Sigmoid):

$$ \underline{a}*{|text{relu}} = |max(0, |underline{z}), |quad |overline{a}*{\text{relu}} = \max(0, \overline{z}) $$

$$ \underline{a}*{|text{sigmoid}} = |sigma(|underline{z}), |quad |overline{a}*{\text{sigmoid}} = \sigma(\overline{z}) $$

- *Explanation:* Because ReLU and Sigmoid are strictly monotonic non-decreasing functions, the minimum input maps to the minimum output, and the maximum input maps to the maximum output.

---

5. Viva / Exam Questions for Aryan

- **Q: What is the primary purpose of Interval Bound Propagation in DeepMind's paper?**
  Answer: DeepMind uses IBP to compute rigorous upper and lower bounds on neural network outputs to prove that the network is robust against adversarial perturbations.
- **Q: How does MTC novelly repurpose IBP for microcontroller compilation?**
  Answer: MTC repurposes IBP as a **compile-time numerical safety verifier**. It propagates bounds through the computation graph to mathematically certify that intermediate activations and gradients will never overflow the $[-1.0, +1.0)$ dynamic range of fixed-point Q15 arithmetic on the ESP32.

Quick Reference Summary Table for Aryan

| Paper No. | Short Title | Primary Problem Solved | Key Technique / Concept | Direct MTC Connection | | :---: | :--- | :--- | :--- | :--- | | **Paper 16** | **TVM** (OSDI 2018) | Manual kernel writing across heterogeneous chips | Multi-level IR (Relay $\rightarrow$ TIR) & operator fusion | MTC lowers JSON AST into `IrGraph` DAG $\rightarrow$ C99 loops | | **Paper 17** | **Glow** (Meta 2018) | Excessive tensor memory consumption | **Liveness analysis** and static buffer overlay | MTC Phase 3 statically overlays non-overlapping buffers | | **Paper 18** | **Autodiff Survey** (JMLR 2018) | Misconceptions about differentiation methods | Duality of Forward vs **Reverse Mode** ($O(1)$ sweeps) | Establishes the mathematical foundation for MTC's backprop | | **Paper 19** | **Tapenade & Enzyme** | Runtime overhead of operator-overloading autograd | **Source-to-source compilation** of adjoint code | MTC emits pure C99 derivative functions ahead of time | | **Paper 20** | **DeepMind IBP** (2018) | Neural network vulnerability & bound verification | **Closed Interval Arithmetic** $[\underline{z}, \overline{z}]$ | MTC's core novelty: compile-time Q15 overflow safety proof |

Notes prepared for Aryan Upadhyay (`2405111530037`) • Department of Computer Science & Engineering • G. L. Baja Group of Institutions, Mathura